アンパック時のストアデータを用いた難読化マルウェア検知機構の提案

鈴木 悠華[1,a)]　小林 良太郎[1,b)]　加藤 雅彦[2,c)]

概要：近年，マルウェアによる被害が増加しており，その対策としてマルウェアの静的解析や動的解析が用いられている．しかし，現状ではおよそ半数のマルウェアが難読化されており，静的解析が困難であるという課題が挙げられる．こうした課題を背景に，難読化されたマルウェアのアンパックの処理に着目した機械学習によるマルウェアの分類方法が研究されている．一方で，これらの研究ではアンパック時のマルウェアの特徴ではなくアンパック処理そのものが学習されてしまっている．そのため，False Positive Rate (FPR) が高い，未知のパッカーに弱いという課題がある．そこで本研究では，ストアデータに着目してアンパック処理を行っているときのマルウェアの特徴を発見し，難読化されたマルウェアの検知機構を提案する．

キーワード：マルウェア，難読化

Proposal for a Detection Mechanism for Obfuscated Malware Using Store Data During Unpacking

Yuka Suzuki[1,a)]　Ryotaro Kobayashi[1,b)]　Masahiko Kato[2,c)]

Abstract: Recently, damage caused by malware has increased, and static analysis and dynamic analysis have been used as countermeasures. However, nearly half of malware is currently obfuscated, making analysis by static analysis difficult. To address this issue, machine learning methods that classify malware based on its unpack process have been studied. However, these methods learn the unpack process itself rather than the characteristics of malware during unpack. Consequently, they suffer from high false-positive rates and poor generalization to unknown malware. In this study, we identify the characteristics of malware during unpack by focusing on store data and propose a detection mechanism for obfuscated malware.

Keywords: Malware, Obfuscation

1. はじめに

近年，サイバー攻撃による被害は深刻化しており，その総数は年々増加している．デジタルアーツが 2026 年 1 月に公開した過去 3 年分のセキュリティインシデント集計によると，国内セキュリティインシデントの総数は 2023 年は 1,195 件であるのに対し，2025 年には 1,782 件まで増加している [1]．その中でもマルウェア感染は 368 件と約 5 分の 1 を占めている．さらに学校・教育機関に関するインシデントを見ると，マルウェア感染のインシデントは 2024 年はたったの 5 件であるのに対し，2025 年は 176 件と大きく増加している．マルウェアとは，ユーザのデバイスに不利益をもたらす悪意のあるプログラムやソフトウェアを総称する言葉である．代表的な例に，ウイルスやワーム，トロイの木馬などが挙げられる．サイバー攻撃，特にマル

[1] 工学院大学
Kogakuin University, Shinjuku, Tokyo 163-8677, Japan
[2] 順天堂大学
Juntendo University, Bunkyou, Tokyo 113-8421, Japan
[a)] j123173@ns.kogakuin.ac.jp
[b)] ryo.kobayashi@cc.kogakuin.ac.jp
[c)] m.kato.ug@juntendo.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ウェアによる被害が急増している今，早く正確なマルウェア検知が必要となっている．現在，多くのマルウェアが難読化されており，Gibert らの研究によると 2019 年から 2020 年で観測したマルウェアの約 60%がパックされている [2]．難読化とは，本来は悪意のあるものではない．知的財産の保護や改ざん防止のために日常的に使用されている技術である．しかし，難読化はマルウェアの解析を困難にしたり，検出を回避したりする目的で悪用されている．難読化はコード変換，パッキング，暗号化層，ポリモーフィックエンジン，メタモーフィック書き換えなどが主流となっている．この中でもパッキングは非常に簡単に行うことができ，多くのマルウェアにおいてパッカーによる難読化が行われている．

パッキングツールは様々な種類があり，特に有名なものは UPX や ASPack というオープンソースのパッカーが挙げられる．パッキングを行う際，パッカーは元のプログラムを圧縮する．その後，圧縮したプログラムを展開するためのコードを作成し，パッキング後のファイルに保存する．この圧縮によってプログラムの構造やデータが変化し，解析が困難となる．

マルウェアの解析手法は，主に静的解析と動的解析という 2 つの手法が挙げられる．静的解析とは，ソースコードやバイナリコード，ファイル構造を解析することで，コードを実行することなくマルウェアの解析を行うことができる解析手法である．コードの実行をしないため安全性が高いことや，実行される可能性のある処理を広く確認できるというメリットがある．一方で，パッキングや暗号化などによる意図的に解析耐性を持たせたものに対して静的解析を行うことは困難である [3][4]．しかし，難読化されると静的解析は役に立たないということはなく，難読化の影響を受けるがマルウェアの検知に利用できる情報もある [5]．

動的解析とは，マルウェアを隔離した環境で実行し，その振る舞いを観察する解析手法である．関数呼び出しの監視や引数の解析，メモリアクセスなどの振る舞いを観察する [3]．そのため，パッキングや難読化に静的解析よりも強いというメリットがある．しかし，実行されなかったコードは観察できず，解析環境の検出や権限昇格によって解析を回避される可能性がある [6]．

このようなことから，パッキングされたマルウェアは静的解析が困難であるため，パッキングされたマルウェアの解析は動的解析が主流である．

アンパック処理とは，パッキングされたマルウェアをメモリ上に展開し，オリジナルコードを実行できるようにする処理のことである．パッキングされたマルウェアを実行すると，元のエントリポイント (EP) から命令を実行するのではなく，パッカーが作成したプログラム展開用のコードが最初に実行される．このコードによってオリジナルコードはメモリ上に展開され，展開が完了するとオリジナルエントリポイント (OEP) へジャンプし，オリジナルコードが実行されていく．

OEP とは，展開処理が完了しメモリ上に展開されたオリジナルコードのうち，最初に実行される命令位置のことである．OEP はオリジナルコードを逆アセンブルする際の開始地点となるため，OEP の特定は非常に重要である．

アンパックの処理に着目したマルウェア検出の研究も存在している．しかし，これらの研究ではパッキングされたマルウェアの特徴やアンパック時のマルウェアの特徴ではなく，パッカーの特徴によってマルウェアの判別が行われてしまっている．そのため，機械学習による False Positive (FP) が出てしまう，未知のパッカーに弱いという弱点がある．ストア命令とは，CPU 内部のレジスタに一時保存されているデータや計算結果をメモリの指定されたアドレスに書き込む命令のことである．このストア命令によってパッキングされたマルウェアはメモリ上に展開されるため，アンパック時のストア命令にマルウェアの特徴が出ると考えた．本研究はストア命令に着目し，アンパック時のストア命令からマルウェアの検出を行うことが目的である．

本論文の構成は以下の通りである．まず第 2 章で関連研究について述べる．第 3 章で提案手法の構造について述べ，第 4 章で評価結果について述べる．それをもとに第 5 章で考察について述べる．第 6 章で結論と今後の課題について述べる．

2. 関連研究

難読化されたマルウェアの分類や OEP の特定には様々な種類が存在している．

Aghakhani らは静的解析特徴量に基づく機械学習が，パックされた実行ファイルに対してどのように機能するかを調査した [7]．静的解析特徴量とは，静的解析で得ることができる一般的なファイル情報や PE ヘッダ情報を指している．Aghakhani らは 3 つの実験を行った．1 つ目の実験では，パックされていない良性ファイルとパックされたマルウェアを用いて作成した分類器でパックされた良性ファイルを分類した．この結果，パックされた良性ファイルに対して 23.40%の False Positive Rate (FPR) を示した．2 つ目の実験では，複数のパッカーを用いた機械学習の分類を行い，どのパッカーに対しても Precision および Recall はともに 99.99%を維持した．このことからパッカーが静的解析特徴量に非常に強い痕跡を残していることが示された．3 つ目の実験では，学習データ中のパックされた良性ファイル割合を高く変化させた時における分類器の性能調査を行った．この実験によって，パック済みの良性ファイルの割合を高くするほど分類器の性能は上昇したということが示された．これらの実験により，パッキングによる静的解析ベースの検知性能の低下，未知のパッカーに対する分類性能の低下，静的解析特徴量の操作による検知器の回

© 2026 Information Processing Society of Japan

避が示された．

Gibert らはパッキングが機械学習検知器にどの程度の影響を与えるのかを調べるために，8 種類のパッカー・難読化ツールが与える影響を調査した [2]．1 つ目の実験では，パッキングされていない良性ファイルとマルウェアで分類器を作成し，パッキングされたマルウェアの検知を行った．5 種類の静的機械学習モデルを使用し実験を行った結果，パッキングされていないファイルに対しては高い性能を示した一方，パッキングされたマルウェアに対しては大きく性能が低下した．これらは特徴量ベース，生バイト列ベース，画像ベースという異なる機械学習モデルすべてで同じ傾向がみられた．実験 2 では，パッキングしたファイルを学習データに入れた場合における分類器の性能について調べた．8 つのパッカーを使用した結果，半分のパッカーに対しては性能の向上が認められたが，もう半分のパッカーに対しては依然として性能の低下が確認された．性能が低下したパッカーでは，暗号化，Import Table 変更，PE 構造変更などが行われており，静的特徴量が大きく変化するパッカーであるという共通点が存在した．実験 3 では，良性ファイルとマルウェアで別のパッカーを用いてその影響を調べた．その結果，検知器は良性ファイルに使用されていたパッカーでパックされた実行ファイルを良性と判断し，マルウェアに使用されていたパッカーでパックされた実行ファイルを悪性と判断する傾向が非常に強かった．実験 4 では，特徴量ベース，生バイト列ベース，画像ベースの機械学習モデルを用いてパッカーの識別を行った．その結果，すべてのモデルにおいて 99%以上の精度でパッカーの識別ができた．これらの実験により，異なるパッカーは実行ファイルの構造に特徴的な変化を与えるため，その違いは静的特徴量から容易に検出できることが示された．

岸部らは解析支援を目的として，パッキングのアルゴリズムに依存しない精度の高い OEP 特定手法の提案を行った [8]．この提案手法は，検体の実行命令系列をメモリ上のアドレス距離によりグルーピングを行う点，各グループがオリジナルコードか否かを機械学習で判定する点という二つの特徴を有している．グルーピングはパッキングされたプログラムの実行命令系列をメモリ上のアドレス距離に応じて行う．複数に分割されたグループはオリジナルコードの命令群または，複合ルーチンの命令群で構成されている．これらのグループに対して最長共通部分列に着目し，オリジナルコードのグループ同士であれば最長一致長が比較的長くなることを用いてオリジナルコードの判定を行い，そのうち最初に実行されたアドレスを OEP としている．これにより，評価実験では未知のパッカー 20 種類中 18 種類に対して OEP の特定を行うことができた．

近年のマルウェアは，仮想マシンやデバッガを検出すると動作を停止するものや，特定バージョンのアプリケーションが存在するときのみ攻撃を行うもの，外部のサーバから追加ファイルをダウンロードするものなどが存在している．これらのマルウェアに対しても解析を行うために，仲小路らは多種環境マルウェア動的解析システム (M3AS) の提案と評価を行った [9]．このシステムはマルウェアを条件の異なる環境で実行し，その挙動を観測する．条件の違いとは，解析エンジン，プラットフォーム，アーキテクチャ，OS, OS 言語，アプリケーションの 6 つである．また，M3AS はネットワーク再現機能を有しており，ダウンローダ型マルウェアの挙動も再現・確認することができる．この結果，M3AS は解析したマルウェアのうち 93%からマルウェアの特徴を検知でき，環境選択型マルウェアのうち 63%から顕現条件が抽出できることを示した．

静的解析と動的解析を組み合わせたマルウェアの解析手法も存在する．従来の動的解析では，選ばれなかった分岐の挙動を観測することができない．泉田らは動的解析と静的解析を組み合わせ，制御フローの解析を行うことによるマルウェアの解析手法について述べた [10]．このシステムは制御部と静的解析部，動的解析部の 3 つから構成されており，制御部は静的解析部と動的解析部を連携させて解析システム全体を駆動させる役割を持つ．また，静的解析部では任意の開始アドレスから静的に解析できる範囲の制御フローグラフ (CFG) を作成し，動的解析部では解析対象となる PE 形式ファイルを実行して，静的解析に必要な実行時情報を収集する．これにより，いくつかのマルウェアにおいて解析が可能であることを示した．また，静的解析と動的解析を組み合わせることにより，それぞれの解析を単体で行った場合よりも広い範囲の制御フローを抽出した．

本研究では，これらの研究では行われていない時系列に沿ったメモリ内のデータを取得するという点で差異を持つ．

3. 提案手法

本研究では，ストア命令に着目しアンパック時の処理からマルウェアの検出を行うことを目的としており，ストア命令のクラスタリングを用いた OEP の特定とアンパック時の処理の特徴からマルウェアの検出を行うものである．

OEP の特定では，マルウェアを実行したときに得られるトレースを用いる．このトレースはストア命令のみを時系列に沿って取得するものであり，得られたトレースを CSV ファイルとして出力する．その後，取得したトレースのストアアドレスを用いたクラスタリングを行う．これによって各クラスタの上書き率や大きさなどからオリジナルコードの候補となる位置を絞る．取得したトレースを時系列順に調べ，プログラムカウンタ (pc) がオリジナルコードの候補に入ったとき，そのクラスタがオリジナルコードであると推定され，その pc の周辺に OEP が存在する可能性があると考えられる．

© 2026 Information Processing Society of Japan

3.1 QEMU

OEP の特定には QEMU 上でマルウェアを実行し，取得したトレースを用いて行う．オープンソースでクロスプラットフォームのエミュレータである QEMU 10.2.0 を用い，CPU を ARM, マシンを versatilepb, ドライブを raspios に設定する．トレースを取得するためのプラグインは，QEMU のプラグインとして存在する execlog.c を基に自ら作成したものを使用する．トレース取得の流れとしては，最初にターミナルで QEMU の起動を自作のプラグインを指定して行う．次に QEMU を起動したターミナルとは別のターミナルを開き，トレースの取得を開始するコマンドの入力を行う．その後 QEMU 内でマルウェアを実行し，ある程度の時間が経過した後にターミナルからトレースの取得を終了するコマンドを入力する．取得したトレースは 10,000 命令ごとにカレントディレクトリに CSV ファイルとして出力される．

本研究では，QEMU でのトレースの取得で表 1 の特徴量を得た．

表 1 特徴量

特徴量	内容
pc	プログラムカウンタ
disas	逆アセンブル
store_vaddr	ストアアドレス
store_size	ストアサイズ
reg	ストアレジスタ
value	ストアデータ

この表における disas とは，命令を逆アセンブルした結果のことである．出力例としては，str r2，[r1，#0x14] や, push {r4，r5，r6，r7，lr}といったものがある．また，store_vaddr はその命令がどこに値をストアするのかということを示しており，store_size は実際にストアされる値の大きさを表している．reg はその命令がどのレジスタを使用したのかを示しており，value は実際にストアされる値のことである．

3.2 クラスタリング

QEMU で取得したトレースを用いてクラスタリングを行う．クラスタリングのプログラムは自作のものを使用する．クラスタリングの流れは，最初に 100 ファイルずつクラスタリングを行い，その後各クラスタリング結果をマージする．クラスタリングではストアアドレスが近いもの同士でクラスタを作成し，マージ後の各クラスタの特徴を調べる．表 2 は，クラスタリングの結果で得られる特徴である．

表 2 クラスタリングで得られる特徴

特徴	内容
idx	クラスタ番号
start	クラスタの開始アドレス
end	クラスタの終了アドレス
size	クラスタ範囲の大きさ
write	ストア回数
total_written	書き込まれた総バイト数
unique	クラスタ内で実際に埋まったバイト数
density	書き込み密度
overwritten	上書きされたバイト数
overwritten_ratio	上書き率
pcs	ストアした pc の種類
first	最初の書き込み位置
last	最後の書き込み位置
file_count	クラスタが出現したファイル数
files	クラスタが出現したファイル名

idx は作成したクラスタの開始アドレスを比べて小さい順につけた番号のことである．start および end はそのクラスタの開始アドレスと終了アドレスのことを指しており，例えば 0x1000 から 0x1f00 までのクラスタがあった場合，start は 0x1000, end は 0x1f00 となる．size はそのクラスタにおける範囲の大きさを表しており，先ほどの例を用いると，size=0xf00 bytes となる．write はそのクラスタに対するストア回数，total_written は書き込まれた総バイト数であり，それぞれ上書きしたストア命令も含める．これに対して unique は上書きを含めないクラスタ内のストアのバイト数であり，density はそのクラスタ内にどれだけ書き込まれているかを表している．また，overwritten は上書きされたバイト数を，overwritten_ratio はどれだけ上書きされているかを表している．pcs はそのクラスタにストアした pc の種類である．例えば 0x1000 と 0x1fff という pc があるクラスタに 1 回ずつストアした場合の pcs は 2 となり，0x1000 が複数回ストアした場合の pcs は 1 となる．first と last はそれぞれそのクラスタに最初に書き込まれたときと最後に書き込まれたときが全トレースの中で何番目かを表している．

4. 評価結果

QEMU 内で Mirai_2 というマルウェアについてトレースを取得し，OEP の推定を行った．Mirai_2 とは，UPX というオープンソースのパッキングツールを用いてパッキングした IoT 系のマルウェアである．図 1 に取得したトレースの一部を示す．左から pc, store_vaddr, value を表している．この図からは，上から 5 命令が同じ pc からのストアであることがわかり，各 value が連続した store_vaddr にストアされている．

これらのトレースを 100 ファイルずつクラスタリングし，各クラスタリング結果のマージを行ったものに

© 2026 Information Processing Society of Japan

pc		store_vaddr		value
0xc054006c	...	0xc06fddec	...	0xcf9f3998
0xc054006c	...	0xc06fddf0	...	0xcfdaa600
0xc054006c	...	0xc06fddf4	...	0x00000021
0xc054006c	...	0xc06fddf8	...	0xc07319b0
0xc054006c	...	0xc06fddfc	...	0xc0039e7c
0xc0039ec0	...	0xcfdaa614	...	0x00000001
0xc0039edc	...	0xcfdaa650	...	0x00000001

図 1 Mirai_2 のトレースデータ例

ついて図 2 に示す．左から idx, start, size, density,

idx	start	size	density	overwritten_ratio	pcs	
0	0x8000	66880	1	0.076753	6	...
1	0x2053c	45302	1	0.995824	354	...
2	0x2bff4	12	1	0.963415	8	...
3	0x2c620	15	1	0.444444	9	...
4	0x30000	29952	1	0	5	...
5	0x5e094	264	0.083333	0.214286	11	...

図 2 Mirai_2 のクラスタリング結果例

overwritten_ratio, pcs を表している．idx=1 の時はクラスタの size は大きいが，overwritten_ratio が高いことからクラスタ内のほとんどで上書きされていることがわかる．一方で，idx=4 の時も idx=1 の時と同様にクラスタの size は大きいが，こちらは overwritten_ratio が 0 であることから上書きは行われておらず，pcs の値が小さいことからほとんどの value が同一の pc からストアされていることがわかる．

アンパック時の処理では，低アドレス領域に多くの書き込みがあり，store_size が小さいものが多かった．また，idx が 0, 1, 4 の部分では，同じ pc によるストア命令が連続的に並んでいる箇所がいくつか存在した．この時，store_vaddr の値は 4 ずつ順に並んでいた．そこで，オリジナルコードは一定の間隔でストアされていくと推定し，この部分を OEP 候補とした．

5. 考察

本研究では，難読化されたマルウェアのアンパック時のストア命令から OEP の位置を推定できることが判明した．ストア命令のトレースを取得し，ストアアドレスをクラスタリングすることによりオリジナルコードの候補を発見した．その後ストア命令の pc が候補の中の一つに入ったことを観測したことにより，候補の中からオリジナルコードの位置を推定することができた．このことによってその pc の周辺に OEP が存在する可能性があると考えられる．本研究では UPX でパッキングされている Mirai_2 というマルウェアのみに対して OEP の推定ができることを示したが，時系列順のトレースデータを取得することで，他のパッキングツールを用いてパッキングされたマルウェアに対しても同様に OEP の位置の推定ができる可能性がある．そのため，今後は UPX でパッキングされた他のマルウェアや別のパッカーによってパッキングされたマルウェアに対してトレースの取得を行い，OEP の推定が可能かどうかを評価する必要がある．また，OEP の推定が可能となったことにより，アンパック処理と考えられる部分のみを全体のトレースから抜き出してその特徴の調査や比較を行うことができると考える．これは UPX によってパッキングされたマルウェアだけでなく，UPX でパッキングされた正常なファイルや別のパッカーによってパッキングされたマルウェアのアンパック処理の比較を行うことで，今後機械学習を用いる際の特徴量の決定に役立つと考える．また，現在はトレースの取得は取得開始のコマンドを入力後，十分な時間が経過した後に取得終了のコマンドを入力している．これでは莫大な量のトレースが取得され，クラスタリングに長時間かかってしまうことになる．よって，パッキングされたファイルはまずメモリに展開されてからオリジナルコードの実行が行われるということを利用して，コマンドを手動で入力してトレースの取得を終了させるのではなく，わずかな時間が経過した後に自動的にトレースの取得を終了するようなコードの改良を行った方が良いと考える．しかし，アンパック処理にどれだけの時間がかかるかは不明のため，トレースの取得時間は慎重に検討する必要がある．また，本研究ではストア命令のみをトレースとして取得しているため，OEP の完全な特定は行うことができず，オリジナルコードのストア命令が実行された時点の pc しかわからないということには注意が必要である．

6. 結論と今後の展望

本研究では，アンパック時のストア命令からパッキングされたマルウェアの OEP を調査した．QEMU を用いて取得したストア命令のトレースに対してクラスタリングを行った結果，OEP の推定を行うことができた．これにより，時系列データを用いた OEP の推定及びアンパック処理と考えられる部分を抜き出すことが可能となった．一方で，現在はトレースを取得して OEP の推定を行い，OEP までのトレースを抜き出すことしか行うことができていない．そのため，今後の課題としてはアンパック時のトレースを調査してマルウェアの特徴を発見し，機械学習を用いて得られた特徴によるマルウェアの検知を行うことが挙げられる．また，機械学習による自動的なマルウェアの分類が実現すれば，LSI 上に分類機構を実装することも期待される．

謝辞
本研究の一部は，JSPS 科研費 26K14827 の支援により行った．

© 2026 Information Processing Society of Japan

参考文献

[1] DigitalArts, “[2026 年 1 月公開] 過去 3 年分の国内セキュリティインシデント集計,” https://www.daj.jp/security_reports/52/ (Accessed 2026-08-12).

[2] D. Gibert, N. Totosis, C. Patsakis, G. Zizzo, and Q. Le, “Assessing the impact of packing on static machine learning-based malware detection and classification systems,” Computers Security, Vol. 156, 2025.

[3] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, “A Survey on Automated Dynamic Malware-Analysis Techniques and Tools,” ACM Computing Surveys, Vol. 44, No. 2, Art. No. 6, pp. 1–42, 2012.

[4] A. Moser, C. Kruegel and E. Kirda, “Limits of Static Analysis for Malware Detection,” in Proceedings of the 23rd Annual Computer Security Applications Conference, pp. 421–430, 2007.

[5] B. Molina-Coronado, A. Ruggia, U. Mori, A. Merlo, A. Mendiburu, and J. Miguel-Alonso, “Light up that Droid! On the Effectiveness of Static Analysis Features against App Obfuscation for Android Malware Detection,” Journal of Network and Computer Applications, Vol. 235, Article 104094, 2025.

[6] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, “Dynamic Malware Analysis in the Modern Era—A State of the Art Survey,” ACM Computing Surveys, Vol. 52, No. 5, Art. No. 88, pp. 1–48, 2019.

[7] H. Aghakhani, F. Gritti, F. Mecca, M. Lindorfer, S. Ortolani, D. Balzarotti, G. Vigna, and C. Kruegel, “When malware is packin ’heat; Limits of machine learning classifiers based on static analysis features,” in Proceedings of the 27th Network and Distributed System Security Symposium, Internet Society, 2020.

[8] 岸部 功太郎, 中村 徳昭, 森井 昌克, 伊沢 亮一, 井上 大介, 中尾 康二, “実行命令列の分類に基づいた汎用的なオリジナルエントリポイント特定手法の提案,” コンピュータセキュリティシンポジウム 2014 論文集, pp. 1148-1155, 2014.

[9] 仲小路 博史, 重本 倫宏, 鬼頭 哲郎, 林 直樹, 寺田 真敏, 菊池 浩明, “多種環境マルウェア動的解析システムの提案および評価,” 情報処理学会論文誌, Vol. 56, No. 9, pp. 1730-1744, 2015.

[10] 泉田 大宗, 森 彰, 二木 厚吉, “展開型静的解析と動的解析を連携させたマルウェア解析手法,” コンピュータ ソフトウェア, Vol. 29, No. 4, pp. 199-218, 2012.

© 2026 Information Processing Society of Japan